Lightweight Predictive Congestion Estimator for TCP: Project Plan

Goal: Compare TCP Tahoe, Reno and CUBIC in NS-3, train a lightweight ML model (Decision Tree / Logistic Regression) that predicts congestion from sender-side measurements, and build a "Predictive TCP" that uses the model to reduce its sending rate *before* packets are lost.

Tech: NS-3.48 (C++), Python 3.14 (scikit-learn, pandas, NumPy, Matplotlib), macOS, VS Code.

---

0. Current Status

Item	Status
CMake + Ninja (Homebrew)	✅ Installed
NS-3.48 in `~/ns-3` (optimized; modules: core, network, internet, point-to-point, applications, traffic-control, flow-monitor)	✅ Built (192 MB)
Project folder + Python venv (`.venv`, scikit-learn 1.9, pandas 3.0)	✅ Ready
NS-3 TCP internals checked (congestion ops, recovery ops, trace sources)	✅ Done
Phase 1: Tahoe/Reno classes, simulation, tracing, 45 baseline runs, validation graph	✅ Done
Phase 2: dataset (106k samples), Decision Tree vs Logistic Regression, C++ export (verified)	✅ Done
Phase 3: TcpPredictive, closed-loop feature selection, 15 runs + ablation; all 9 PRD 10.3 criteria met	✅ Done
Phase 4: all graphs, final table, results page, README, `run_all.sh` (verified reproducible)	✅ Done
Phase 5: Vegas + BBR baselines, 30 runs, delay-vs-loss graph; Predictive beats BBR on all metrics, Vegas has fewer drops	✅ Done
Phase 6: delay-trend features, horizon comparison, model chosen inside NS-3; drops –88%, RTT –24% / –34% vs Reno	✅ Done
Phase 7: controller tuned on seeds 1–3 (β 0.8, MinWindow 4); drops –91% / –90%, RTT –26% / –37%, fairness 0.96, single flow 9.7 Mbps	✅ Done
Phase 8: buffer sweep (gains hold at 20–100 packets, 5/5 seeds) + competition with CUBIC (Predictive gets only 23%)	✅ Done
Phase 8b: competition detector; share vs CUBIC 23% → 36% (unseen seeds 22% → 35%), no cost at 50 packets; false alarms with a 100-packet buffer (documented)	✅ Done
Phase 9: Report + presentation (format to decide)	⬜ Next

Phases 5–9 are described in section 12.

---

1. What Must Be Done First (in order)

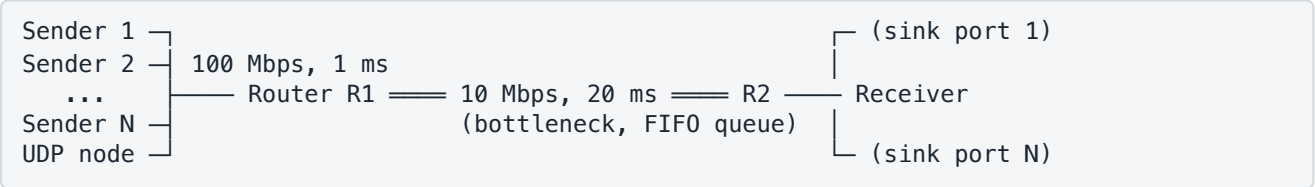
1. **Write the custom TCP variants** (NS-3 has CUBIC and NewReno, but **no Tahoe and no classic Reno**):
   - `TcpTahoeRecovery`: on 3 duplicate ACKs, cwnd is set to **1 segment** (no fast recovery). On exit, cwnd stays at 1 segment so the sender **slow-starts** again.
   - `TcpRenoRecovery`: fast retransmit plus fast recovery (cwnd = ssthresh, inflated by duplicate ACKs). A **partial ACK halves the window again**, which reproduces Reno's known weakness with multiple losses in one window.
   - Congestion control for both: `TcpLinuxReno` (Reno AIMD: +1 segment per RTT, halve on loss).
   - CUBIC: built-in `ns3::TcpCubic`.
2. **Build the simulation program** (`tcp-predictor.cc`) with the dumbbell topology and the 3 congestion levels (Section 2).
3. **Add per-flow tracing** that writes one CSV row per flow every 100 ms (Section 3).
4. **Run a single test simulation** to check: traces are not empty, the queue fills, losses appear, and the CSV columns are correct.
5. **Run the baseline experiments:** 3 variants × 3 levels × 5 seeds = 45 runs.

Only after step 5 can the ML work start, because the model needs this data.

---

2. Network Setup (Simulation Design)

Dumbbell topology:



- Base RTT ≈ 44 ms; bandwidth-delay product ≈ 38 packets.
- **Bottleneck queue:** FIFO (`FifoQueueDisc`), **50 packets**. The device queue is set to 1 packet so all queuing is visible in the queue disc.
- **TCP settings, the same for all variants:** segment size 1448 B, **SACK on**, **minimum RTO 1 s** (RFC 6298), send/receive buffers 4 MB (so the window is never limited by the buffer).
- **Why SACK is on:** NS-3's CUBIC breaks without SACK (measured 4.66 vs 9.48 Mbps for one flow, with 10,315 needless retransmissions). Tahoe's and Reno's window behaviour comes from the custom recovery classes, so it is unchanged.
- **Why the minimum RTO is 1 s:** with 200 ms it equals the receiver's delayed-ACK timer, and Tahoe's one-segment window then timed out on every packet and never recovered.
- **Recovery type:** classic recovery for CUBIC and Predictive; custom recovery for Tahoe and Reno.

Congestion levels:

Level	Traffic	Average load
Low	3 on/off TCP flows at 2 Mbps (exponential on/off, mean 1 s)	~30%
Medium	6 on/off TCP flows at 3 Mbps	~90%, bursty
High	8 long-lived (bulk) TCP flows + 2 Mbps on/off UDP	Saturated

- **Run setup:** 60 s of simulated time per run, flows start at random times between 0 and 2 s, and seeds 1–5.
- **Variants compared:** Tahoe, Reno, CUBIC and (later) Predictive. All flows in a run use the same variant.

---

3. Data to Collect (CSV, every 100 ms per flow)

Column	Source
`time`, `flow`	Simulation clock, flow ID
`srtt_ms`	Socket `RTT` trace (smoothed RTT)
`min_rtt_ms`	Minimum smoothed RTT seen so far
`cwnd_bytes`	Socket `CongestionWindow` trace
`thr_mbps`	Bytes received by that flow's sink in the interval
`retx`	Socket `Retransmission` trace (count in the interval)
`loss_events_0.5s`	Entries into recovery or RTO (`CongState` trace) in the last 0.5 s
`queue_pkts`, `queue_cap`	Bottleneck queue disc `PacketsInQueue` (maximum in the interval)
`drops`	Bottleneck queue disc `Drop` trace (count in the interval)

Also recorded per run: an end-of-run FlowMonitor summary (per-flow lost packets, delay, throughput).

Output files: `results/raw/samples_<variant>_<level>_s<seed>.csv` and `results/raw/flowmon_<variant>_<level>_s<seed>.csv`.

---

4. Machine Learning

Features (8): only things the sender can measure itself. Queue length lives on the router, so a real sender cannot read it.

Feature	Meaning
`rtt_ratio`	srtt / min RTT (how much queuing delay has built up)
`rtt_grad`	Change in srtt since the last interval / min RTT (is the queue growing?)
`est_queue_pkts`	cwnd × (1 − minRTT / srtt), the sender's own estimate of its packets sitting in the queue (Vegas-style)
`cwnd_seg`	Congestion window in segments
`thr_mbps`	Delivery rate of the flow
`cwnd_rel`	cwnd ÷ its maximum over the last 2 s (position in the saw-tooth)
`loss_recent`	Loss events in the last 0.5 s
`loss_2s`	Loss events in the last 2 s

Labels, from the real bottleneck queue, looking **0.5 s into the future** (so the model *predicts*, rather than describes, congestion): - **High:** any drop, or the queue reaches ≥ 80% full in the next 0.5 s - **Medium:** the queue reaches ≥ 40% full - **Low:** otherwise - **Binary version:** congested = High

Training:

- **Data split:** train on seeds 1–3 and test on seeds 4–5, so test runs are never seen in training (no leakage).
- **Models:** Decision Tree (maximum depth 4–5, so it stays tiny) vs Logistic Regression (scaled features).
- **Metrics:** accuracy, precision/recall/F1 per class, confusion matrix, and **early-warning time** (how many ms before each actual drop the model first predicted High).
- **Export:** convert the decision tree to C++ `if/else` code (`predictive-model.h`) so it can run inside NS-3.

Outputs: `results/model/` (metrics, confusion matrices, tree diagram, exported header).

---

5. Predictive TCP (Proposed Mechanism)

`TcpPredictive` extends `TcpLinuxReno` and uses Reno's loss recovery, so any difference from Reno comes from the predictor: 1. **Every 100 ms, compute the features** from its own ACKs (smoothed RTT, minimum RTT, cwnd history, loss events). 2. **Run the exported decision tree.** The final tree has depth 2 and 4 leaves (2 comparisons per prediction). 3. **Act on the prediction:** - **Low:** normal Reno growth. - **Medium:** grow at **25% of Reno's rate** (`MediumGrowth = 0.25`). - **High: reduce cwnd by β = 0.85** (at most once per RTT, only while not already recovering from a loss) and stop growing. 4. **Keep Reno's normal loss response** (halving) as a safety net. 5. **Minimum window:** below **4 segments** the predictor never acts (`MinWindow = 4`).

Design changes found in closed-loop testing (all documented in the report):

- **The deployed model uses only network-state features:** RTT ratio, RTT trend, estimated queue, cwnd relative to its recent maximum, and loss events in the last 0.5 s and 2 s. Throughput and absolute cwnd were removed: once the model controls the sending rate, it reads its own slowdown as congestion and keeps slowing down (a feedback loop).

- **Why the minimum window exists:** at 1 segment the receiver's delayed ACK inflates the RTT to ~240 ms. That looks like congestion and kept the window frozen forever.
- **Slow growth instead of a freeze:** freezing on Medium gave the fewest drops but unfair sharing (Jain 0.78). 25% growth restores fairness (0.99).

Results (5 seeds, against Reno):

- **Medium:** drops –40%, RTT –3.0%, goodput 99.9%.
- **High:** drops –61%, RTT –0.6% (–7.4% vs CUBIC), goodput 100%, fairness 0.988.

Ablation: freeze vs 0.25 vs 0.5 growth, no proactive cut, stronger cut (β = 0.7), and no reaction to Medium (`results/ablation_summary.csv`).

---

6. Graphs and Comparison

1. **Line graphs over time, for one run per level:** throughput, srtt, cwnd and bottleneck queue length for each variant.
2. **Bar charts (mean ± standard deviation over seeds):** throughput, average RTT, packet loss, retransmissions and Jain fairness index, for each variant and level.
3. **ML graphs:** confusion matrix, feature importance, decision tree diagram, a histogram of early-warning time, and Decision Tree vs Logistic Regression scores.
4. **Final comparison table:** Tahoe vs Reno vs CUBIC vs Predictive.

Output folder: `results/graphs/`.

---

7. Final Deliverables

Deliverable	Location
NS-3 source (variants + simulation)	`ns3/` (copied into `~/ns-3/scratch/tcp-predictor/` to build)
Python code (dataset, training, export, graphs)	`analysis/`
Raw results + model outputs	`results/`
Graphs	`results/graphs/`
README (setup, how to run everything)	`README.md`
One-command runner	`run_all.sh`
Project report	`docs/` (format to be decided)
Presentation	`docs/` (format to be decided)

---

8. Project Folder Structure

```
TCP-Congestion-Predictor/
├── ns3/
│   ├── tcp-predictor.cc        # simulation: topology, scenarios, tracing
│   ├── tcp-variants.h/.cc      # TcpTahoeRecovery, TcpRenoRecovery, TcpPredictive
│   └── predictive-model.h      # generated from the trained decision tree
├── analysis/
│   ├── build_dataset.py        # merge CSVs, compute features + future labels
│   ├── train_model.py          # Decision Tree vs Logistic Regression, metrics
│   ├── export_tree.py          # tree → C++ if/else header
│   └── make_graphs.py          # all comparison graphs + tables
├── results/{raw,model,graphs}/
├── docs/PROJECT_PLAN.md        # this file
├── run_all.sh
└── README.md
```

---

9. Task Checklist

Phase 1: Simulation (first)

- ☑ `tcp-variants.h/.cc`: Tahoe and Reno recovery classes
- ☑ `tcp-predictor.cc`: topology, levels, CLI options (`--variant --level --seed --simTime`)
- ☑ Per-flow 100 ms CSV tracing + FlowMonitor summary
- ☑ Test run and sanity check (cwnd sawtooth, queue fills, Tahoe cwnd drops to 1 segment)
- ☑ 45 baseline runs

Phase 2: ML

- ☑ `build_dataset.py` (features + 0.5 s-ahead labels; check class balance)
- ☑ `train_model.py` (Decision Tree vs Logistic Regression, confusion matrix, early-warning time)
- ☑ `export_tree.py` → `predictive-model.h`

Phase 3: Predictive TCP

- ☑ `TcpPredictive` class using the exported tree
- ☑ 15 Predictive runs + compare; tune β if needed (plus ablation study)

Phase 4: Results and Write-up

- ☑ `make_graphs.py` (all graphs + final table + `results/RESULTS.md`)
- ☑ `README.md` + `run_all.sh` (full pipeline in ~2 min, identical results on rerun)

Phases 5–9: see the checklists in section 12.

---

10. Risks and Honest Limitations (to mention in the report)

- **Tahoe and Reno are reimplemented** as NS-3 recovery classes, because NS-3 ships only CUBIC and NewReno-style recovery. The behaviour is very close but not identical to historical implementations.

- **Simulation, not real hardware:** results depend on the chosen topology, queue size and traffic mix.
- **Labels depend on thresholds:** the 40% and 80% queue cut-offs will be stated clearly, and a sensitivity check is a good extra.
- **Trade-off:** a more cautious sender lowers loss and delay but can lose some throughput. This is reported honestly, not hidden.
- **Disk space:** each run writes small CSVs (a few MB in total), and NS-3 uses ~200 MB.

---

11. Decisions Still Open

1. **Deadline** (sets how much tuning and polishing to do).
2. **Report and presentation format** (Word + PowerPoint, PDF, or other).
3. **Thresholds and β:** start with 40%/80% and β = 0.85, then adjust after the first results.

---

12. Improvement Phases (5–9)

Phases 1–4 produced a working project. Phases 5–8 fix the three questions an examiner is most likely to ask, and Phase 9 writes it up:

Likely question	Answered by
"Vegas and BBR already avoid congestion using RTT. How is yours different?"	Phase 5
"Hasn't your tree just learned a loss-counting rule?"	Phase 6
"Why is the RTT at high load no better than Reno's?"	Phase 7
"Does it only work in your one setup?"	Phase 8

Order:

- **Phase 5 first:** the baselines don't depend on anything else, and they show where the current version stands.
- **Phase 6 before Phase 7:** the TCP variant is built on the tree, so the model has to be final first.
- **Phase 8 after Phase 7:** the robustness tests should test the final version.

Rules for every phase:

- **Tuning uses seeds 1–3 only.** Seeds 4–5 stay untouched for the final numbers, so improvements are not overfitted.
- **Keep the current results.** Before Phase 5, `results/` is copied to `results/v1/` so the report can show before and after.
- **Keep the feedback-loop rule.** The model uses only network-state features, never the sender's own throughput (finding 5 in the README).
- **No overclaiming.** If a target is missed, the attempt and the reason are reported.

Phase 5: Stronger Baselines (Vegas, BBR) — about half a day

Goal: compare Predictive TCP with the two best-known TCPs that react to delay instead of loss.

☑ Copy the current results to `results/v1/`

☑ Add `--variant=vegas` (`TcpVegas`) and `--variant=bbr` (`TcpBbr`); both are built into NS-3.48. BBR needs pacing turned on.

☑ 30 new runs (2 variants × 3 levels × 5 seeds) + single-flow validation traces

☑ Add both variants to `compare_variants.py`, `make_graphs.py` and the final table

☑ Write a short "where Predictive TCP wins and loses" comparison for the README

Result (✅ done):

- **vs BBR:** Predictive TCP is better or equal on every metric: drops –82% (medium) and –92% (high), RTT –11% and –7%, fairness 0.99 vs 0.85 at high load. BBRv1 overflows the shallow buffer.
- **vs Vegas:** Vegas has far fewer drops at medium load (9 vs 419) and fewer at high load (277 vs 627). Predictive TCP has lower RTT at high load (5/5 seeds) and fairer sharing (0.99 vs 0.88).
- **Vegas alone reaches only 5.3 Mbps** of 10, because delayed ACKs inflate its RTT samples.
- **New graph:** `tradeoff_delay_loss.png`. The earlier results and the model are unchanged (checked against `results/v1/`).

Done when: all 6 variants are in the final table and graphs, and the comparison is written up honestly (including any metric where Vegas or BBR is better).

Phase 6: Stronger ML Model — a few hours

Goal: show the model learns more than a loss count, and raise prediction scores.

☑ New queue-trend features: RTT change over the last 0.5 s and 1 s, RTT vs its 1 s average, time since the last loss

☑ Compare prediction horizons of 0.2 s, 0.5 s and 1.0 s

☑ Compare "loss features only" vs the full model, so the gain from delay features is measured

☑ Feature-importance graph

☑ Add the new features to `TcpPredictive::Evaluate`, re-export the tree, re-run the C++ check (must be 100% identical)

☑ *Added:* test 5 candidate models inside NS-3 on seeds 1–3 (`select_model.py`) and deploy the best by a fixed rule

Done when: scores are reported for each horizon, the deployed tree uses at least one delay-trend feature, and the C++ export matches scikit-learn exactly. **Realistic target:** macro-F1 from 0.62 to about 0.65–0.70. It cannot go much higher, because a sender cannot foresee another flow switching on.

Result (✅ done):

- **Offline:** the full feature set at 0.2 s scores best (macro-F1 0.68, High recall 0.85). At the deployed 0.5 s horizon, macro-F1 stays about 0.61–0.63 for every feature set.
- **Inside NS-3,** the delay-only model (rules use `rtt_ratio` and `rtt_vs_avg`) was by far the best controller, so it is deployed. Results vs Reno over all 5 seeds:
- **Medium:** drops –88%, RTT –24%, goodput 97.9%.
- **High:** drops –88%, RTT –34%, goodput 99.7%, fairness 0.93.
- **Held-out seeds 4–5** show the same gains, so the result is not overfitted.
- **PRD 10.2:** 3 of 4 targets met. Macro-F1 (0.61) is still below 0.70.

- **Finding:** the best offline model is not the best controller. Early warning matters more than macro-F1.

Phase 7: Delay-Aware Predictive TCP — about half a day

> **Status after Phase 6:** the main target below (high-load RTT lower than Reno in at least 4 of 5 seeds) is already met: 5/5 seeds, –34%. Phase 7 now focuses on tuning on seeds 1–3: β (the ablation suggests 0.7 may be better), and fairness at high load (0.93).

Goal: lower delay at high load, not only fewer drops.

☑ React when queuing delay starts rising (not only after losses): done by the Phase 6 delay tree
☑ Grid search **on seeds 1–3 only**: 6 values of β × 3 minimum windows (Medium growth was left out because the ablation showed it no longer matters)
☑ Re-run the 15 Predictive runs and the ablation with the chosen settings
☑ Check that goodput and fairness still pass the PRD criteria
☑ *Added:* single-flow goodput check in the selection rule

Done when: at high load, RTT is lower than Reno's in at least 4 of 5 seeds, drops stay at least 30% below Reno, goodput stays at least 90% of Reno, and the Jain index stays at least 0.9. If these targets can't all be met together, the trade-off is reported.

Result (✅ done): all targets met. - **Chosen:** β = 0.8 (cut 20% on High) and MinWindow = 4, by a fixed rule on seeds 1–3. The scripts are `tune_controller.py` and `tuning_grid.png`. - **Results vs Reno over all 5 seeds:** - **Medium:** drops –91%, RTT –26%, goodput 98.0%. - **High:** drops –90%, RTT –37% (5/5 seeds), goodput 99.6%, fairness 0.96 (up from 0.93). - **Held-out seeds 4–5:** drops –92% / –90%, RTT –26% / –37%. - **Single flow:** 9.7 Mbps (Reno 9.0). - **Lesson (README finding 13):** the first rule checked only medium and high load, and chose β = 0.5. That left a lone flow at 87% of Reno's goodput. A single-flow check (≥ 97%) was then added, and the rule chose β = 0.8.

Phase 8: Robustness Tests — a few hours

Goal: show the results hold beyond one setup.

☑ Buffer sweep: 20, 50 and 100 packets (about 0.5×, 1.3× and 2.6× the bandwidth-delay product of ~38 packets), all 6 variants
☑ Mixed competition: 4 flows of a variant + 4 CUBIC flows sharing the bottleneck (new `--mix` option), measuring each group's share; Predictive, Reno, Vegas, BBR, and CUBIC itself as a control
☑ Graphs: `robustness_buffer.png`, `mixed_competition.png`
☑ Update the Limitations section with what was found

Done when: both tests have a table and graph, and the README states clearly whether Predictive TCP loses bandwidth against CUBIC.

Result (✅ done):

- **Buffer sizes:** at 20 and 100 packets, Predictive TCP still has 86–96% fewer drops and a lower RTT than Reno in every seed. The RTT gain grows with the buffer: –17% at 20 packets and –50% at 100, at high load. Goodput stays ≥ 97.7% of Reno and fairness ≥ 0.92.
- **Competition with CUBIC (4 vs 4, high load):** Predictive TCP gets only **23%** of the bandwidth. For comparison: Vegas 35%, Reno 45%, CUBIC itself 48%, BBR 94%. This is the classic delay-based

weakness. A possible fix (switching to Reno behaviour when its own cuts don't shrink the queue) is described in README finding 15 but not implemented.

- **Check:** a mixed run of Reno with Reno is byte-identical to a plain Reno run, so `--mix` has no side effects.

Phase 8b: Competition Detector (added after Phase 8)

Goal: reduce the loss of bandwidth to CUBIC found in Phase 8 without giving up the main results.

☑ Detector in `TcpPredictive`: if the lowest RTT over a window stays above a threshold × base RTT (the queue never drained), assume a loss-based competitor and behave like Reno. Stay so for a hold time after the last evidence.
☑ Tune window × threshold × hold (3 × 3 × 3, plus off) on seeds 1–3 (`tune_competition.py`), with a rule fixed in advance:

- **Limits:** the Phase 7 limits, plus at least 50% fewer drops and 10% lower RTT than Reno with only Predictive flows.
- **Then:** the share against CUBIC closest to 50%.

☑ Earlier phases pinned to "detector off", so their choices reproduce. Ablation runs use "detector off" so they measure the predictor alone.
☑ Robustness runs repeated with the detector; detector-off comparison runs added.

Result (✅ done):

- **Chosen setting:** window 2 s, threshold 1.4, hold 10 s.
- **Share against CUBIC:**

Seeds	Without detector	With detector
1–3 (training)	23.9%	36.5%
4–5 (unseen)	21.9%	34.9%
All 5	23%	36%

- **Cost with the 50-packet buffer:** none. It never fires there, so the main results are byte-identical.
- **Trade-off:** settings that reach 40–44% also fire with no competitor present, because at high load the queue never fully drains, and they lose most of the gains.
- **Weakness:** with a 100-packet buffer at high load it false-alarms in 3 of 5 seeds, and the alarm feeds itself. Drops fall to –62% instead of –87% vs Reno, and RTT to –24% instead of –50%. This is documented as README finding 16.

Phase 9: Report + Presentation — about 1 day

□ Decide the format (Word + PowerPoint, PDF + PowerPoint, or a faculty template)
□ Report: Abstract, Introduction, Background (Tahoe, Reno, CUBIC, Vegas, BBR), Design, ML Model, Predictive TCP, Results, Robustness, Limitations, Conclusion, References
□ Slides: about 12–15
□ Final `run_all.sh` check and README numbers updated

Done when: both files are ready to submit.

Time Summary

Phase	Effort
5. Stronger baselines	~½ day
6. Stronger ML model	a few hours
7. Delay-aware Predictive TCP	~½ day
8. Robustness tests	a few hours
9. Report + presentation	~1 day

If time is short: do Phases 5, 7 and 9, and skip 6 and 8. Phase 7 can use the existing RTT-ratio feature.